

OVERVIEW

YOLO & TRANSFORMER

MAIN4 - Han, Mohamed Saad, Adrien, Victor

YOLO

Classifiers

- Image = 3D tensor ($W \times H \times C$)
- Pixel = RGB values
- CNN = kernels (K_R , K_G , K_B)

→ Values of importance

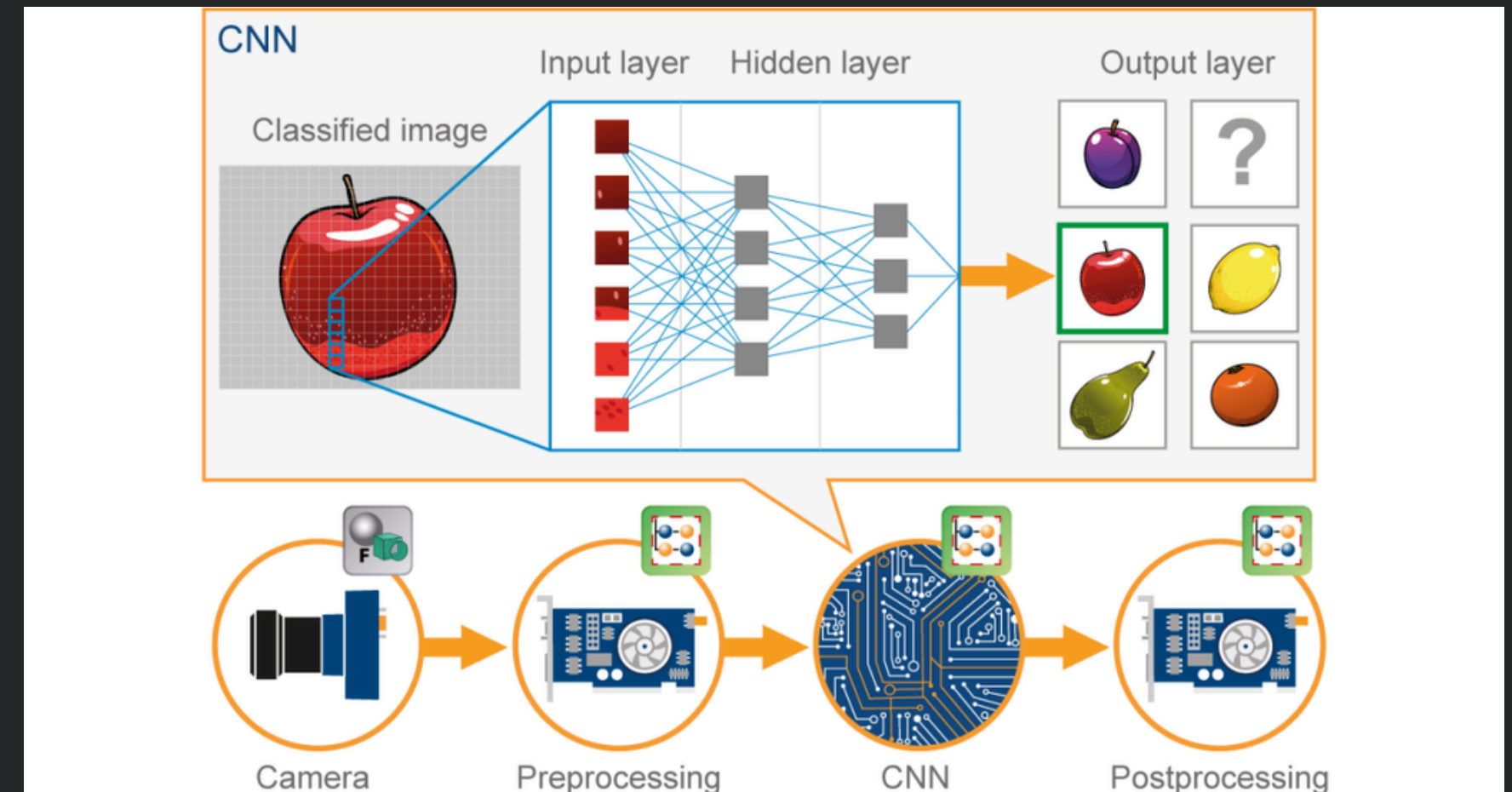
- Output formula

→ Feature map

- Activation function (ReLU, Sigmoid)

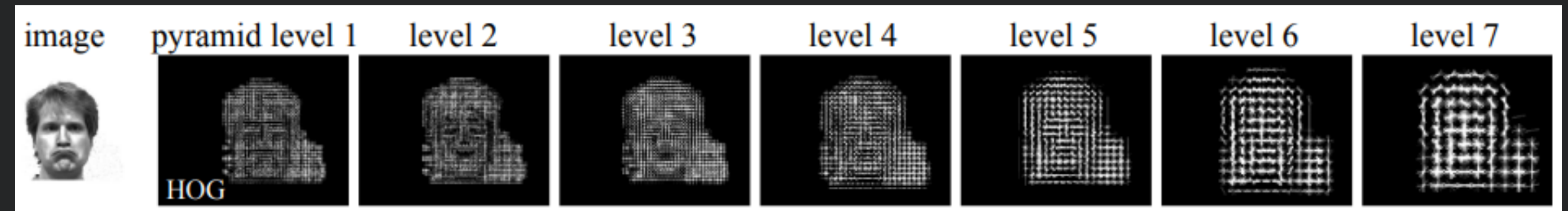
→ Nonlinearity

=> Predicts a class label



Layer	What the pixel values represent
Input layer	Physical light intensities (0–255 per channel)
Conv Layer 1	Weighted combinations of R/G/B values : edge maps, texture maps
Conv Layer 3–5	Abstract features : curves, corners, blobs
Deep layers	Semantic parts : “dog ear”, “wheel”, “face”
Final layer	Class logits : “dog”, “car”

YOLO

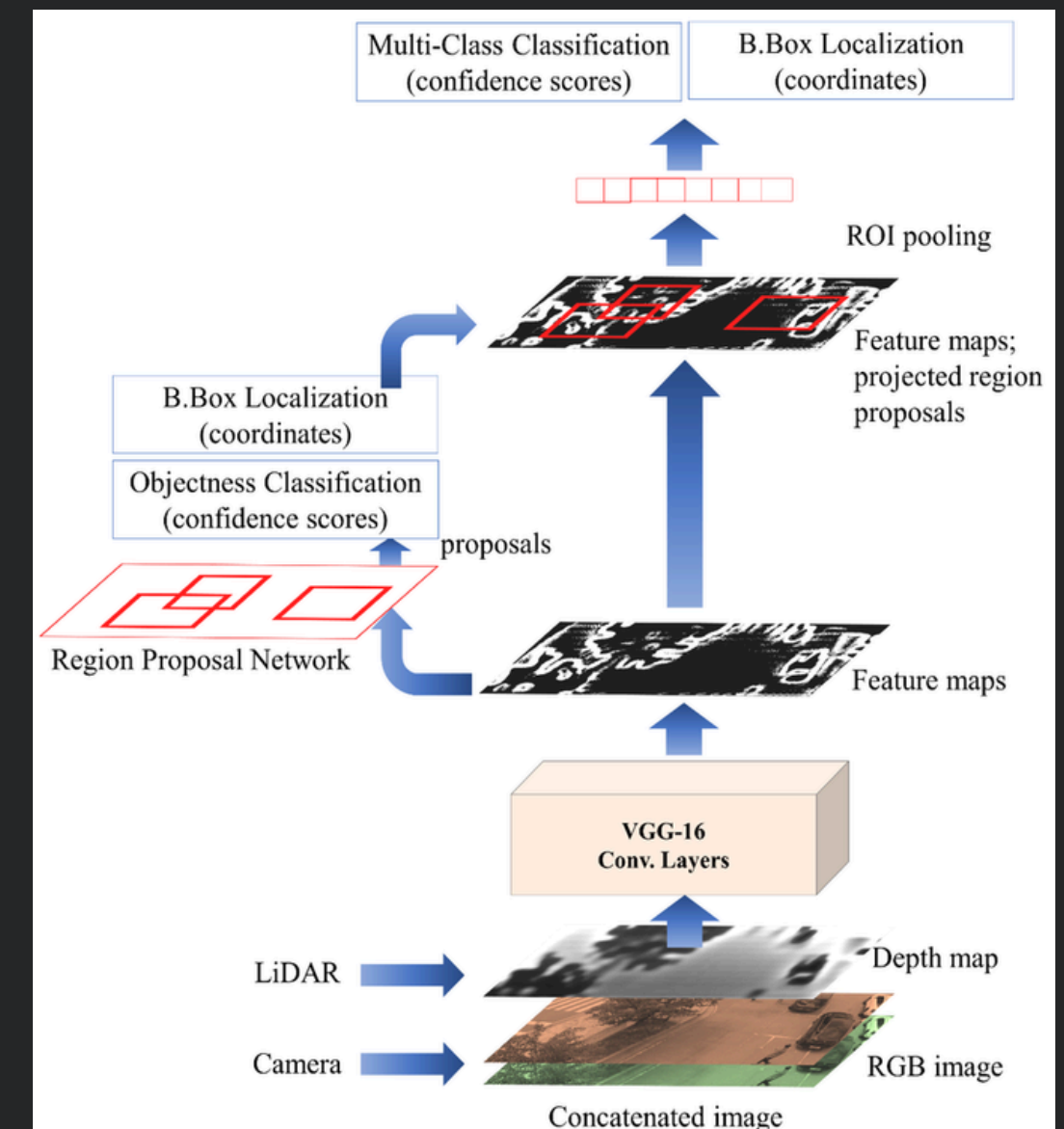


<https://arxiv.org/pdf/1409.5403>

Image detection methods

- DPM (Deformable Parts Models)
 - Sliding window
 - Many classifiers on each window
- R-CNN (Region-based CNN)
 - Region proposal (selective search)
 - Bounding boxes
 - Classifier
 - Post-processing

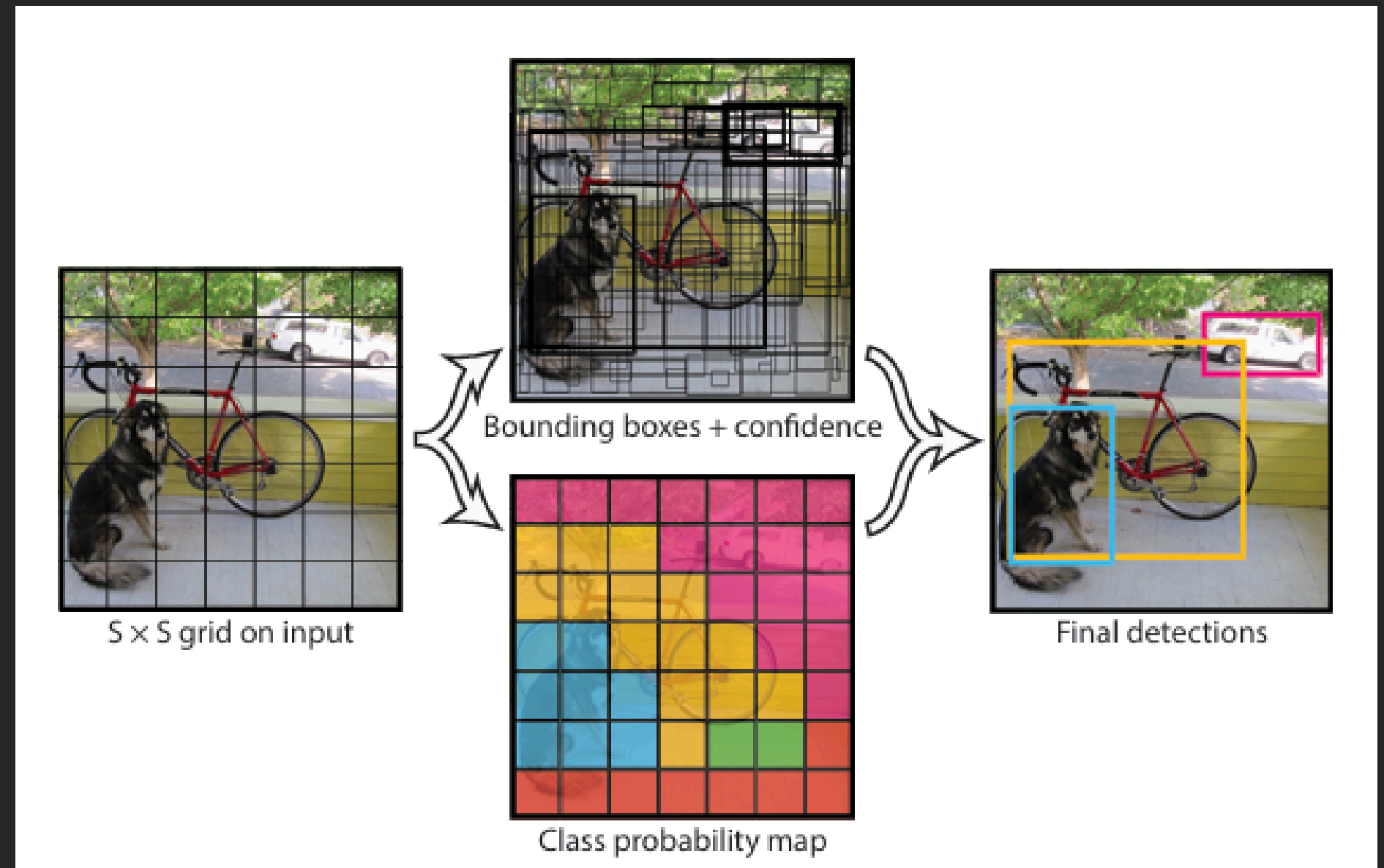
https://www.researchgate.net/publication/349712442_RGDiNet_Efficient_Onboard_Object_Detection_with_Faster_R-CNN_for_Air-to-Ground_Surveillance



YOLO

Principle

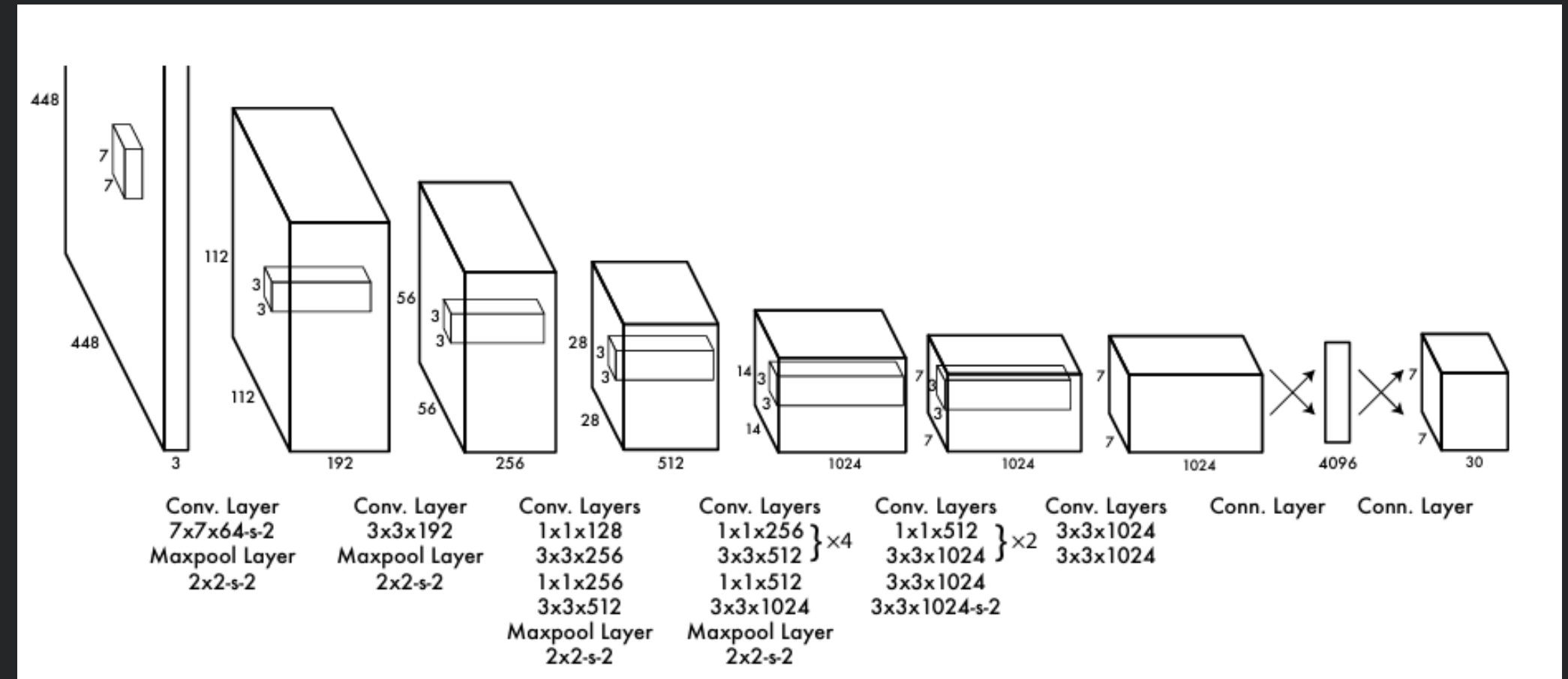
- Single regression problem
→ 1 single CNN does all in once
- Output:
→ $7*7*30$ tensor of predictions
→ $S*S$ grid division
→ B bounding boxes ($B=2$ for Yolo1)
→ C class probabilities ($C=20$ for Yolo1)



YOLO

CNN - Architecture

- 24 convolutional layers
+ 2 fully connected layers



- Input = 448*448*3 tensor (Height*Width*RGB Channels)
- Final output = 7*7*30 tensor of predictions
 - 20 class probabilities
 - 2 bounding boxes * 5 predictions : x,y,w,h,c

YOLO

$$Y_k(i,j) = \sum_{x=0}^{\text{Height}-1} \sum_{y=0}^{\text{Width}-1} \sum_{z=0}^{\text{Channels}-1} K_k(x,y,z) \cdot X(i+x,j+y,z) + b_k$$

CNN - 1st convolutional layer

- 64 7*7*3 kernels (stride = 2, padding = 3 for Yolo1)
- Sliding over the input image to extract features : (448+5)*(448+5) pixels

- Output formula
- 64 feature maps

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{448 - 7 + 2 \times 3}{2} \right\rfloor + 1 = \lfloor 223.5 \rfloor + 1 = 223 + 1 = 224$$

- Output = 224*224*64 tensor

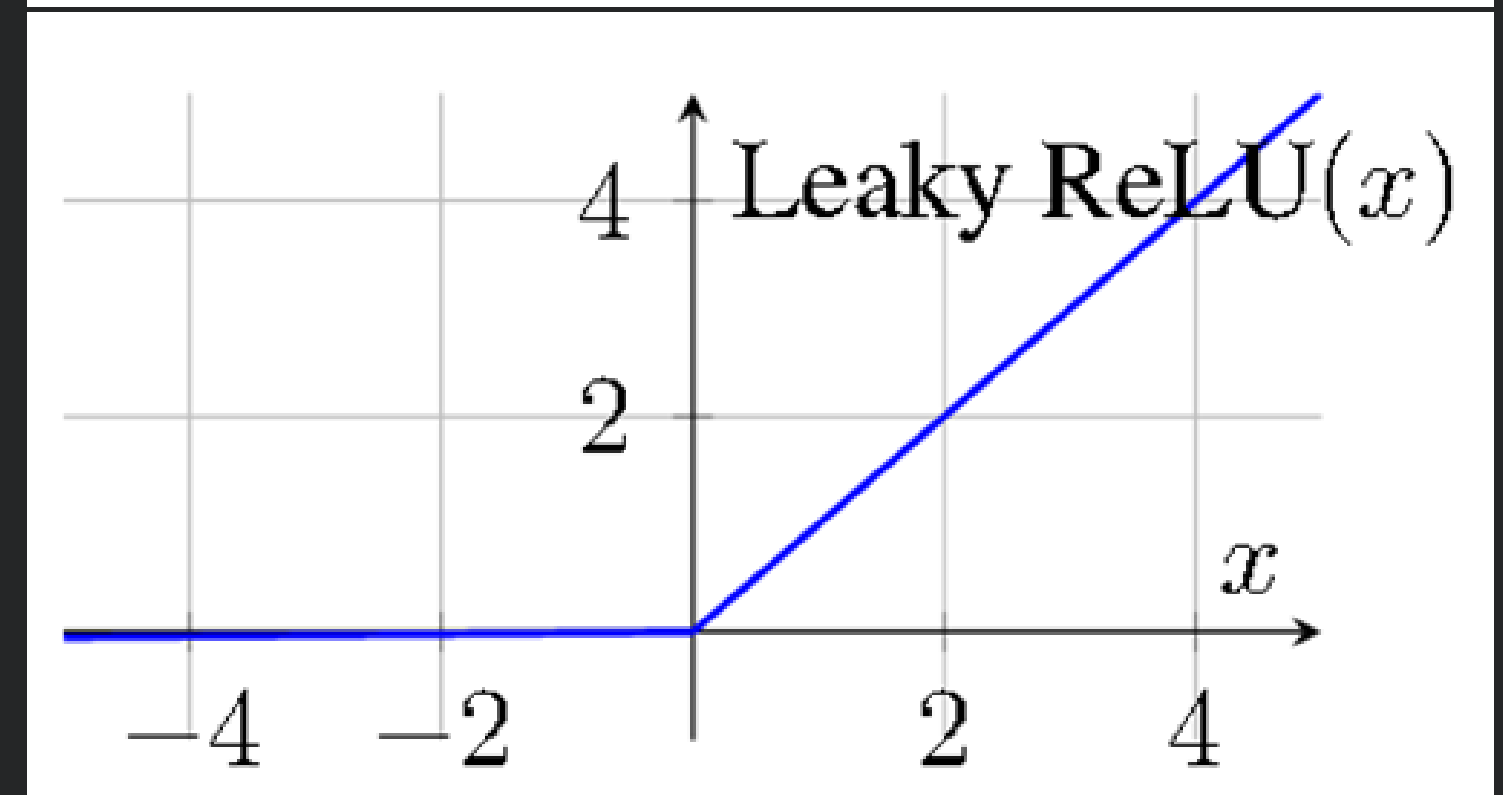
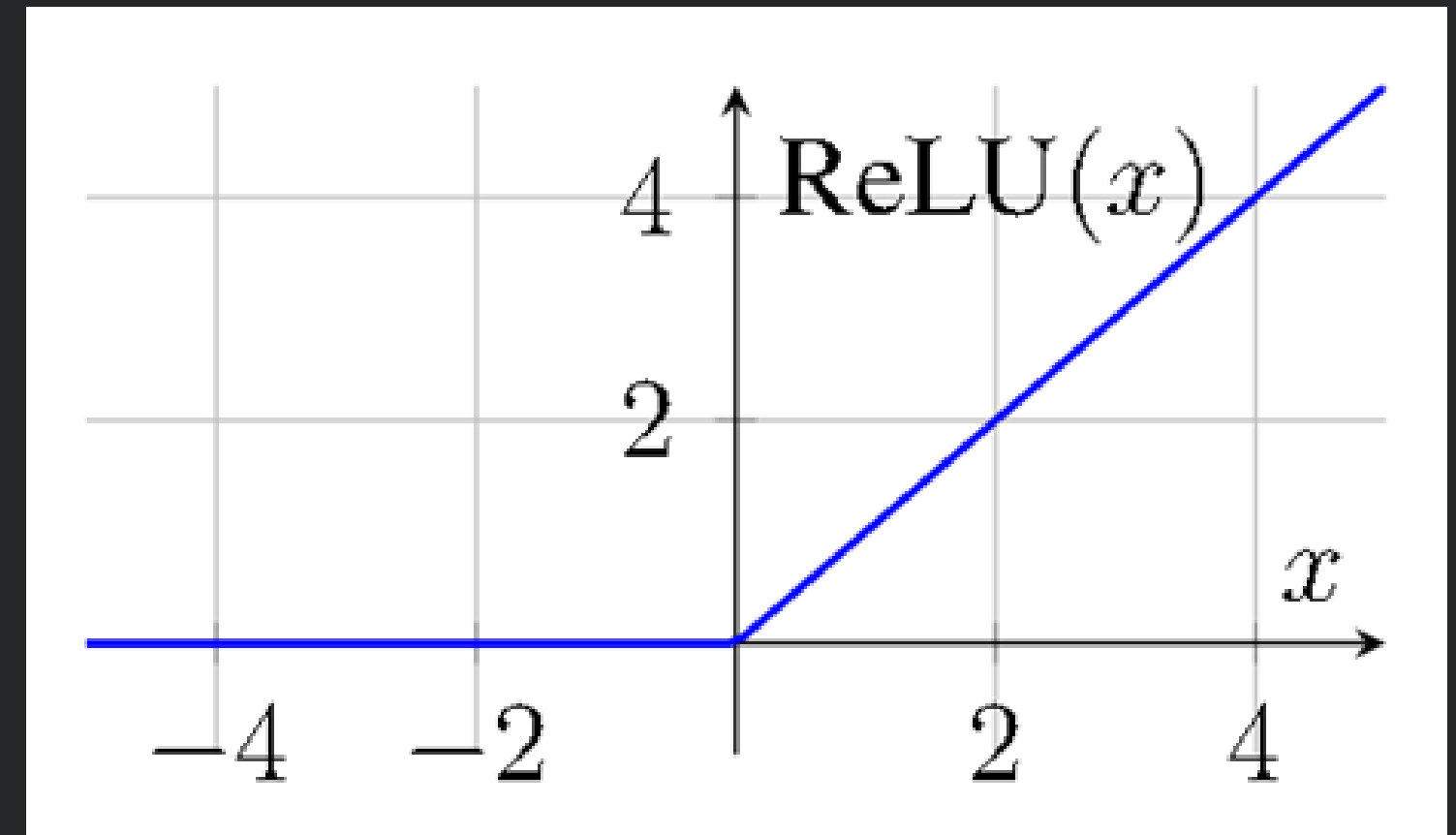
YOLO

CNN - 1st convolutional layer

- Leaky ReLU activation function
- Introducing non-linearity

$$f(x) = \begin{cases} x & x > 0 \\ 0.1x & x \leq 0 \end{cases}$$

- Does not modify the output format

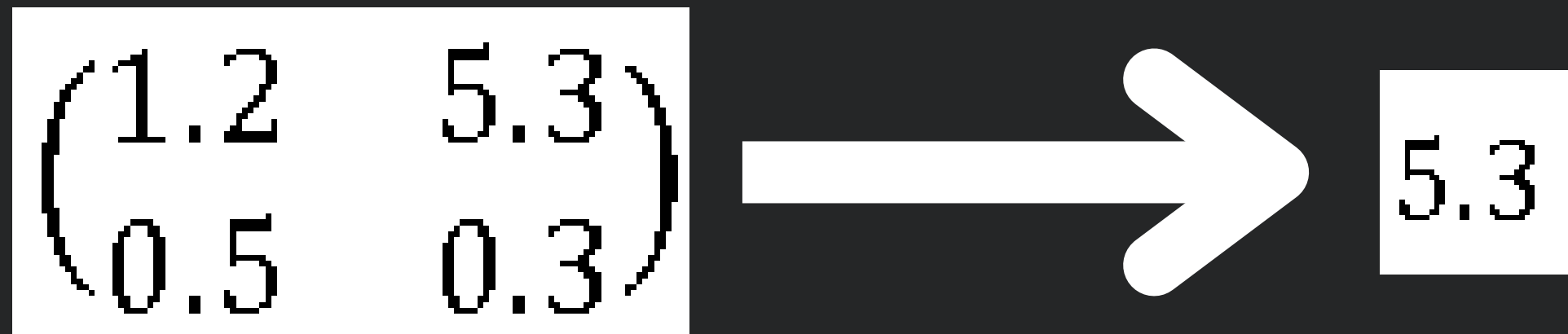


YOLO

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{224 - 2 + 2 \times 0}{2} \right\rfloor + 1 = \lfloor 111 \rfloor + 1 = 111 + 1 = 112$$

CNN - 1st max pooling layer

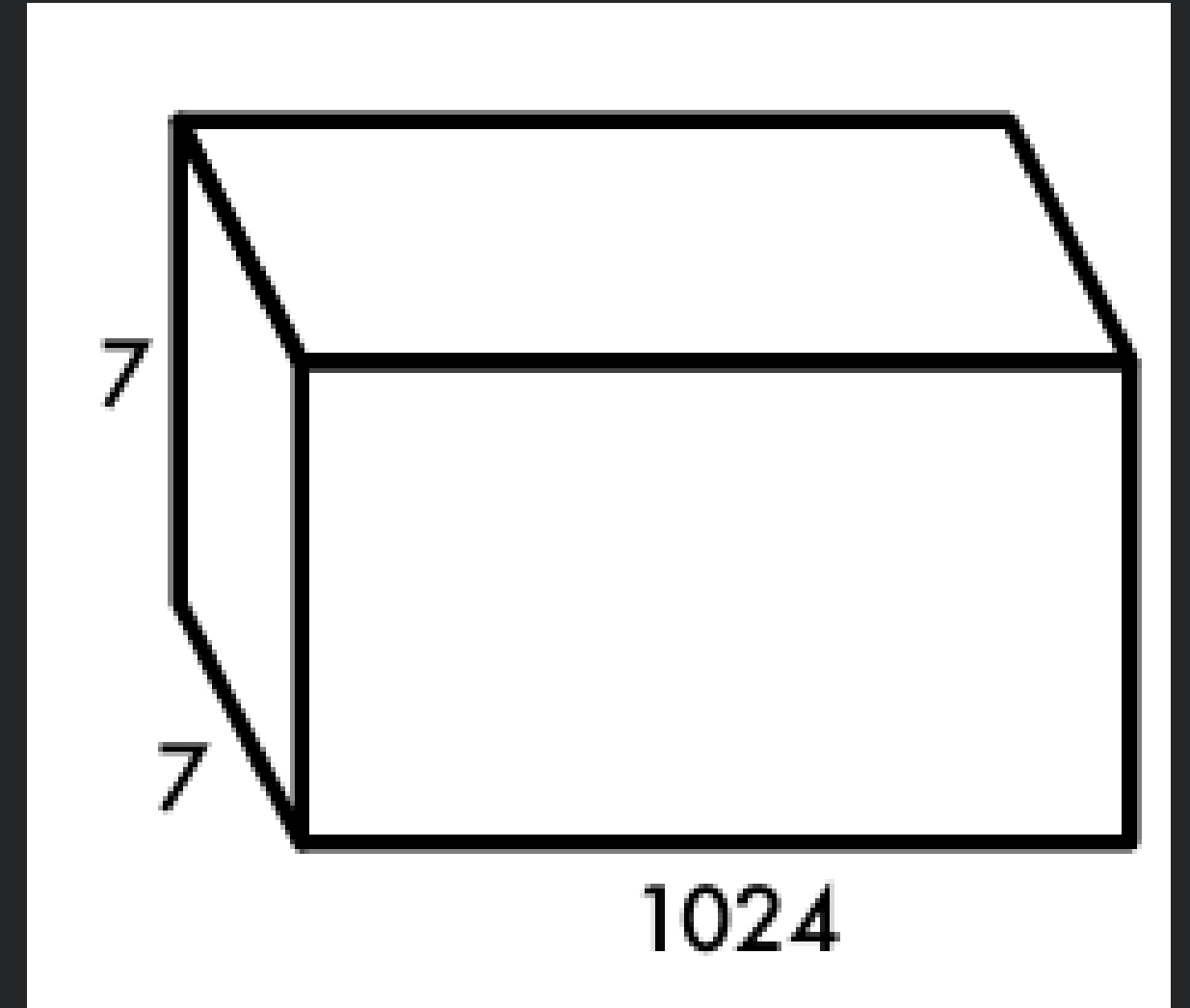
- MaxPooling layers
→ Reduce the spatial dimensions (width and height)
- Output = 112*112*64 tensor



YOLO

Last convolutional layer

- Same principle
 - Output: $7*7*1024$ tensor
- Spatial grid : “Where things are”
- Each cell’s 1024 values : “What is there”



YOLO

$$y_i = \sum_{j=1}^{50176} w_{ij}x_j + b_i \quad \text{for } i \in \{1, \dots, 4096\}$$

1st fully connected layer

- Flattens the feature map into a 1D-vector with dimensions:

$$7*7*1024 = 50176$$

- 4096 neurons with 50176 weights each
- Output formula
- Leaky ReLU function

YOLO

[IMAGE OF FAST YOLO CNN]

Fast YOLO

- Fewer convolutionnal layers
→ 9 instead of 24
- Fewer filters/kernels
→ No precise information
- No information about Fast Yolo's CNN !

YOLO

Performance (Pictures)

Real-time detectors

→ Faster & more accurate in time

- YOLO1: 45 FPS, 63.4 mAP
- FAST YOLO: 155 FPS, 52.7 mAP
- DPM (100Hz): 100 FPS, 16.0 mAP
- R-CNN: 6 FPS, 53.5 mAP

→ Hard to improve Less Than real-time detectors in time-accuracy

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

YOLO

Performance (Artwork)

Yolo generalizes better than other detection systems

- YOLO1: 59.2 AP, 45 AP
- R-CNN : 54.2 AP, 26 AP
- DPM : 37.8 AP, 32 AP

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$F1 = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$$

	VOC 2007	Picasso		People-Art
	AP	AP	Best F_1	AP
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

TRANSFORMERS

- Avant 2017 : domination de RNN, LSTM, GRU pour le NLP
- Ces modèles traitent les tokens(inputs) séquentiellement → limite la parallélisation
- Attention existait déjà, mais toujours en complément d'un RNN
- Objectif : abolir totalement la récursivité

TRANSFORMERS

Notion importante : Architecture

- Architecture Encoder-Decoder
- Encodeur : Va changer l'entrée en vecteur exploitable par le décodeur (embedding)
- Décodeur : Va à partir de l'embedding va pouvoir traduire vers la langue ciblée
- Ici le décodeur récupère à chaque fois la sortie précédente du décodeur

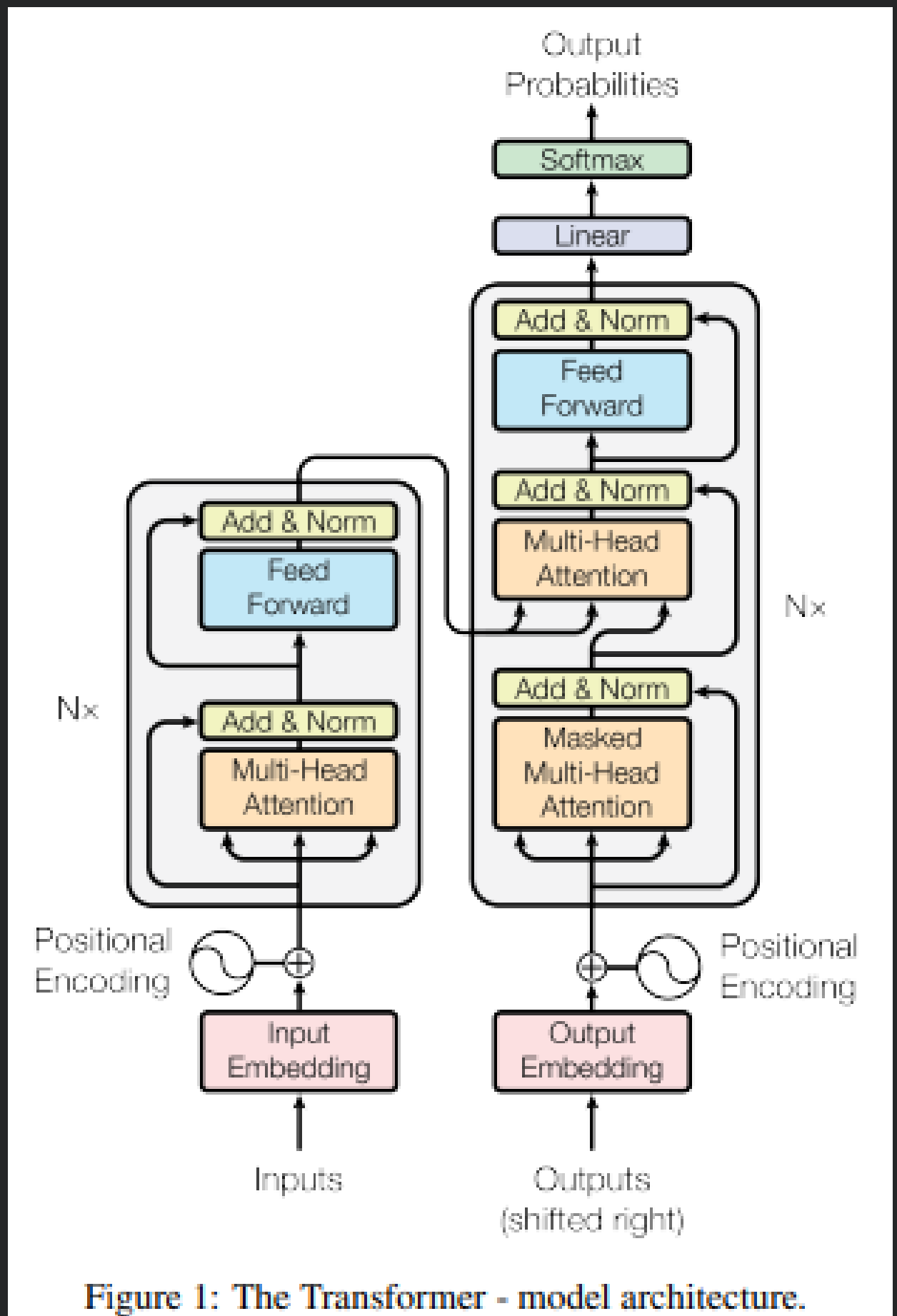


Figure 1: The Transformer - model architecture.

TRANSFORMERS

Architecture d'un Transformer

- Architecture Encoder-Decoder empilée (N=6 couches)
- Chaque bloc = Attention + Feed Forward + Residual + Norm
- Encodeur = Self-Attention uniquement
- Décodeur = Self-Attention + Cross-Attention

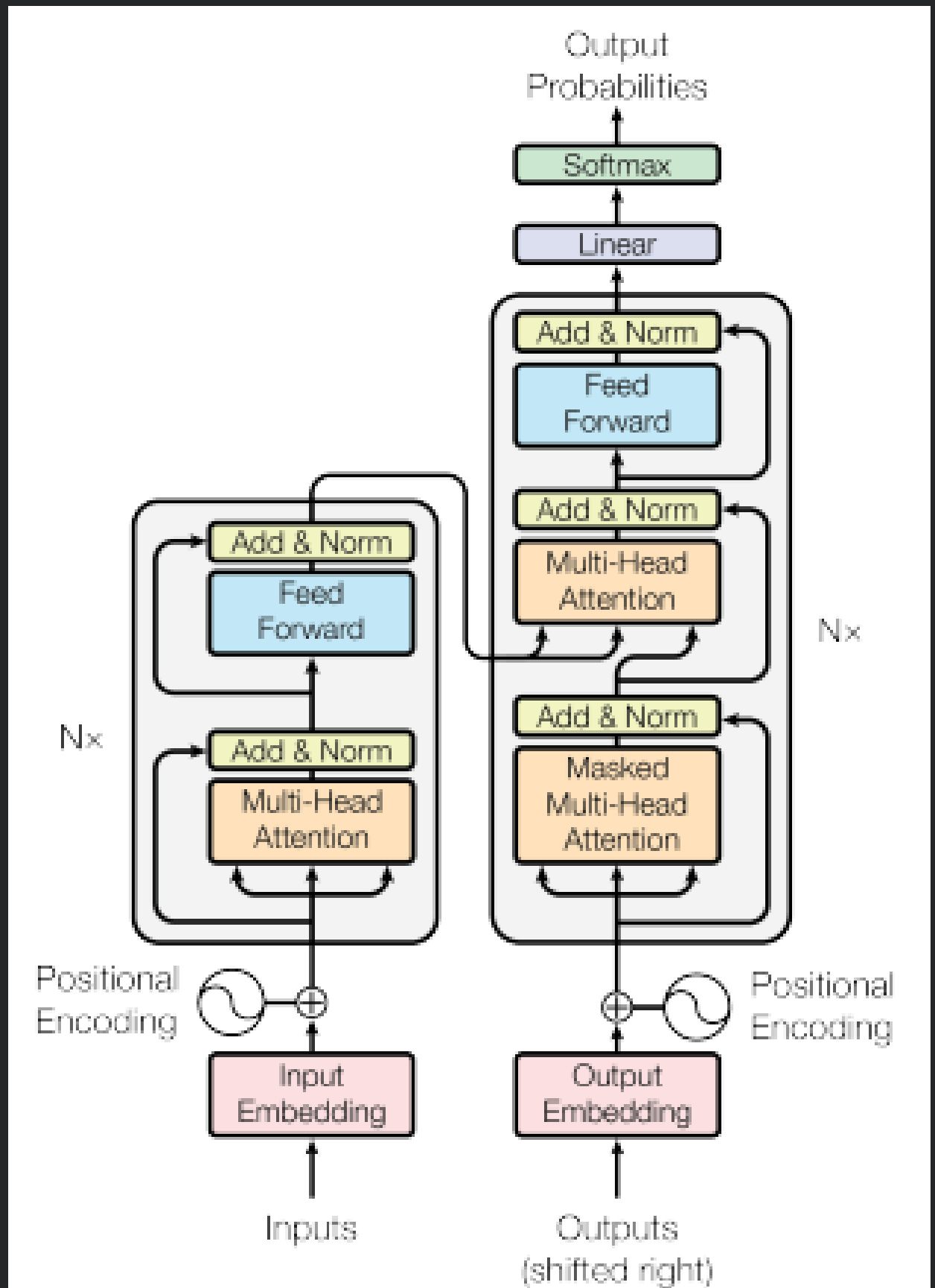
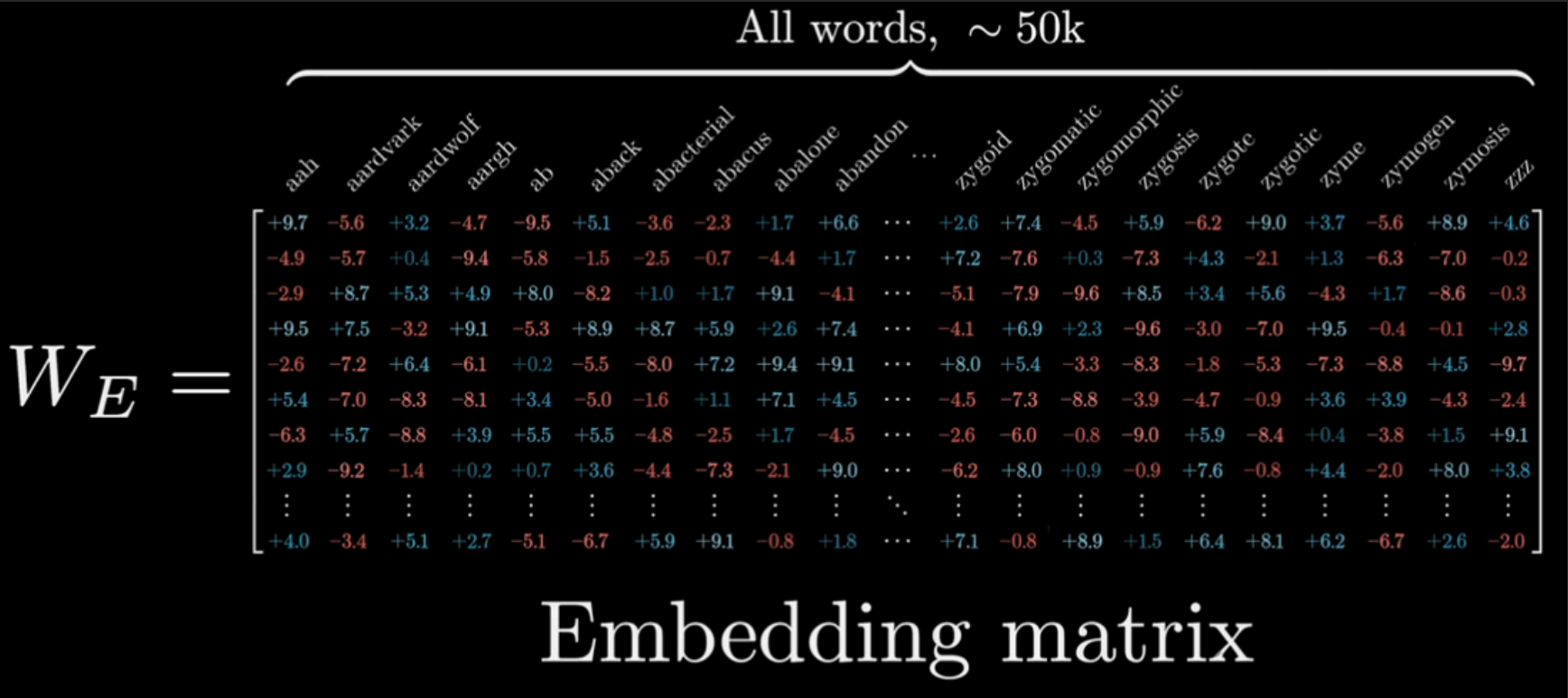


Figure 1: The Transformer - model architecture.

TRANSFORMERS

Embedding

- Les embedding sont rentrés au hasard au début, mais ils évoluent au fur et à mesure que le modèle s'entraîne



Attention in transformers, step-by-step | Deep Learning Chapter 6 - 3Blue1Brown

TRANSFORMERS

Positional Encoding

- Le modèle n'ayant ni RNN ni convolution, on doit injecter la position des tokens.
- On ajoute des encodages positionnels aux embeddings (même dimension).
- Ici sinus/cosinus car ils sont continus et ne saturent pas
- Le principe consiste à calculer la différence entre deux positions via une simple transformation linéaire

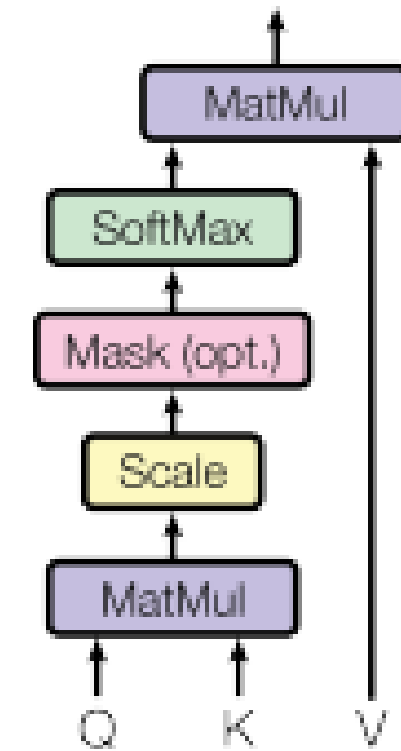
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

TRANSFORMERS

Notion importante: Attention

- Qu'est-ce que ça affecte : Q,K,V
- Représente les dépendances entre chaque input
- On divise par $\sqrt{d_k}$ → stabilise gradients
- Softmax crée distribution d'attention normalisée

Scaled Dot-Product Attention

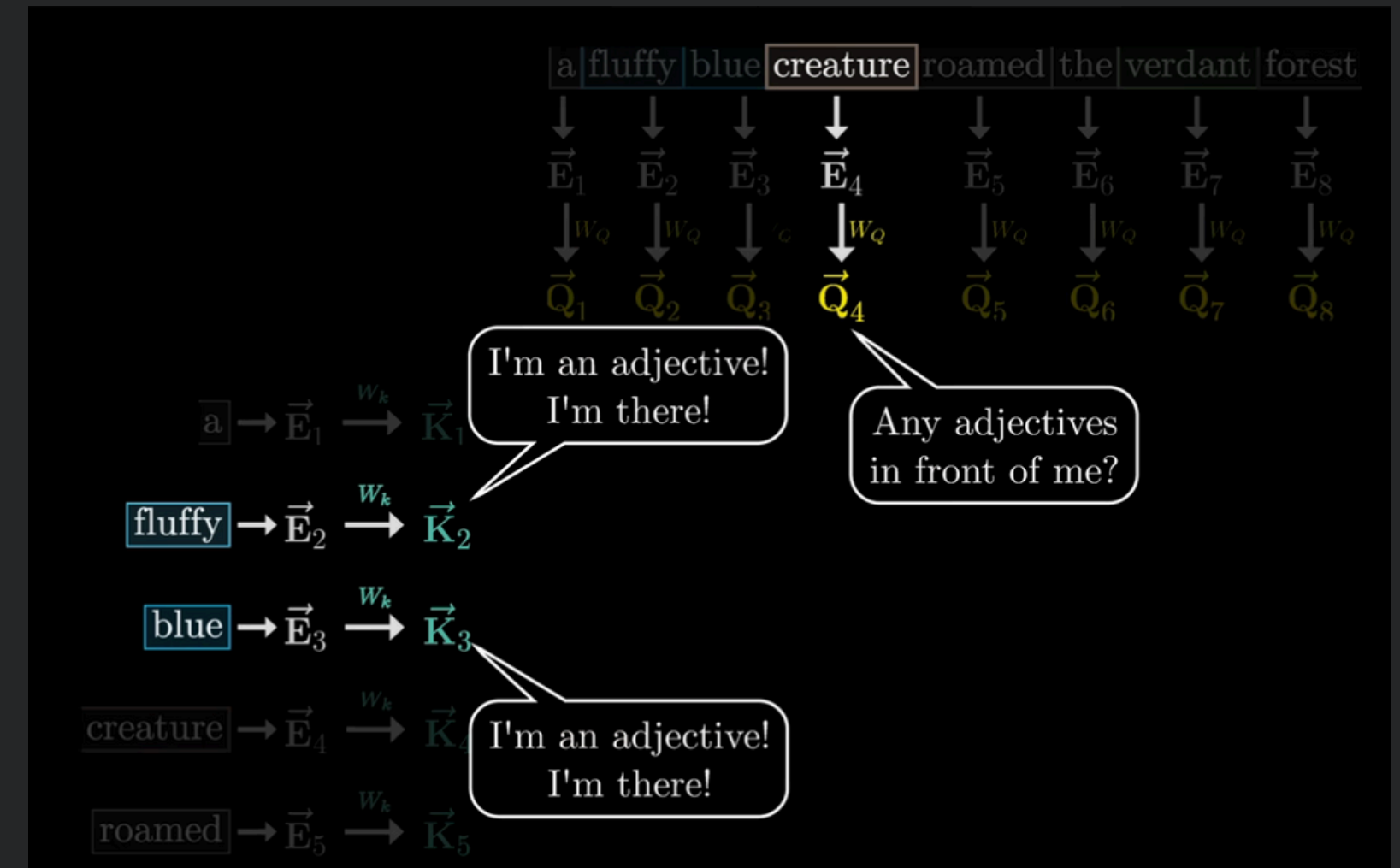


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

TRANSFORMERS

Notion importante: Attention

- Le Query = “question” encodée dans une dimension plus réduite que l’embedding.
- Les Keys représentent les “réponses” aux questions posées par les Query.



Attention in transformers, step-by-step | Deep Learning Chapter 6 - 3Blue1Brown

TRANSFORMERS

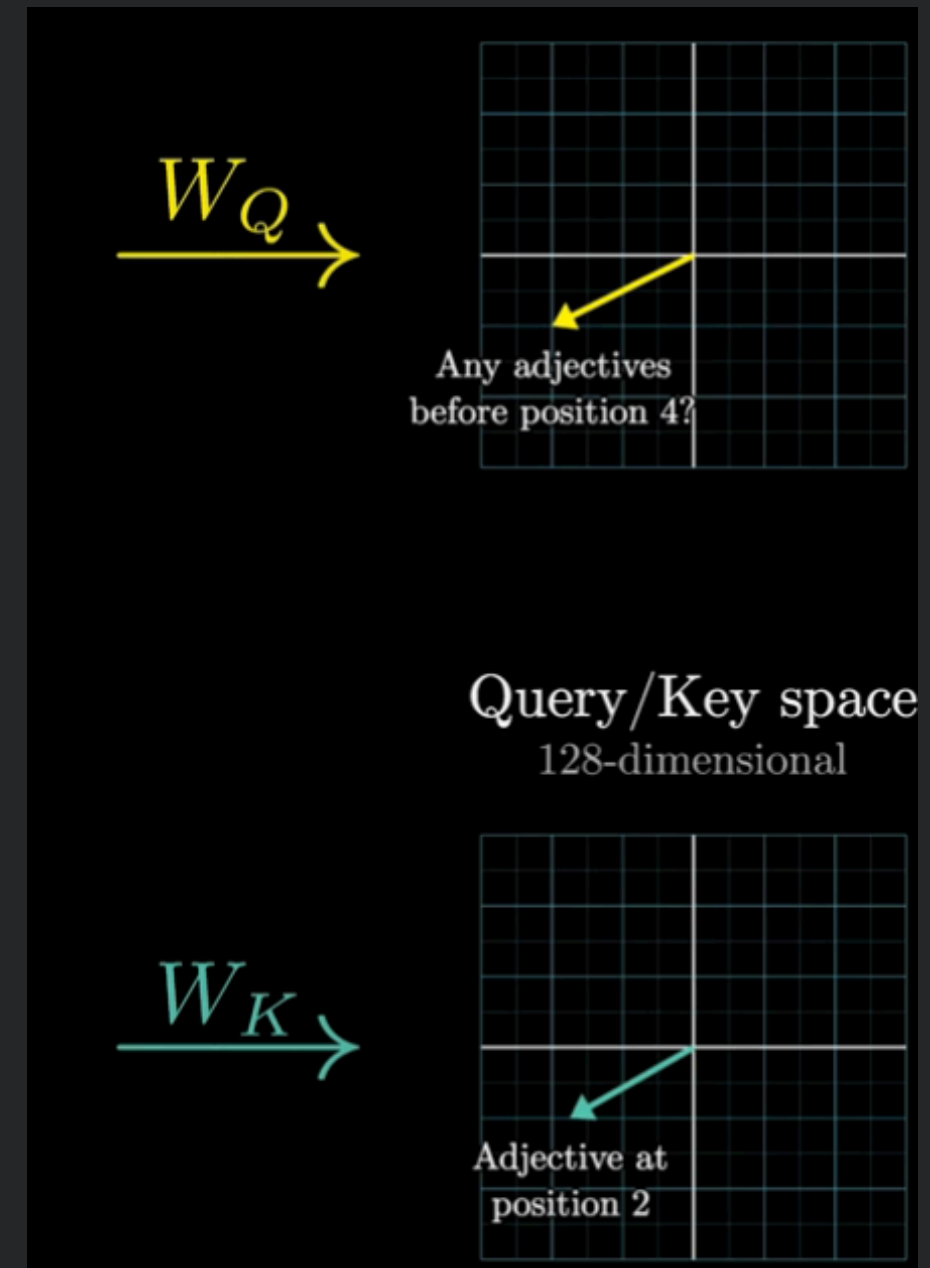
Notion importante: Attention

- On peut imaginer que :
projection de W_k sur un adjectif associé à un nom

≈

projection de W_Q sur ce nom.

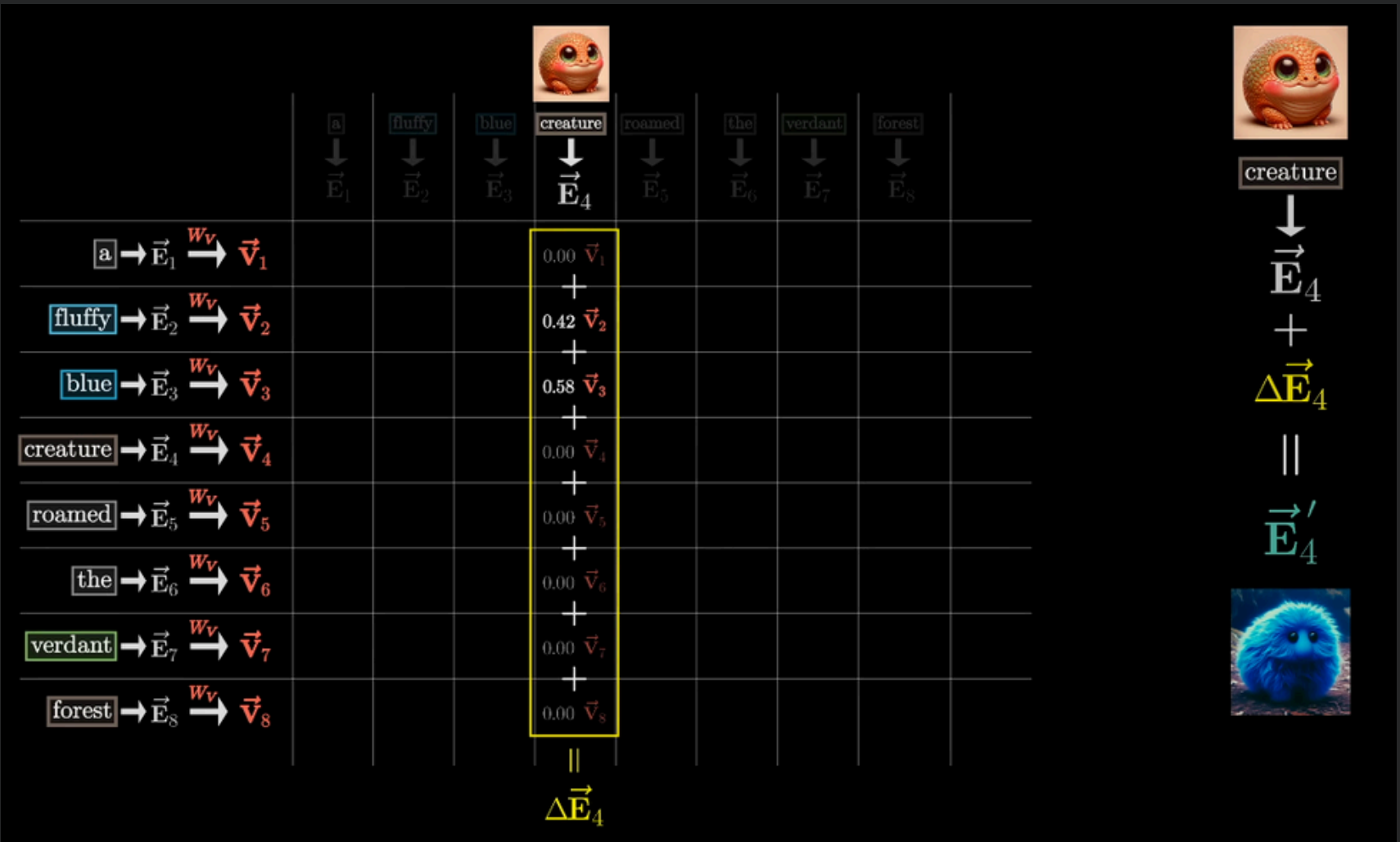
- Et donc plus le produit scalaire entre QK est élevé, plus l'attention est grande.



TRANSFORMERS

Notion importante: Attention

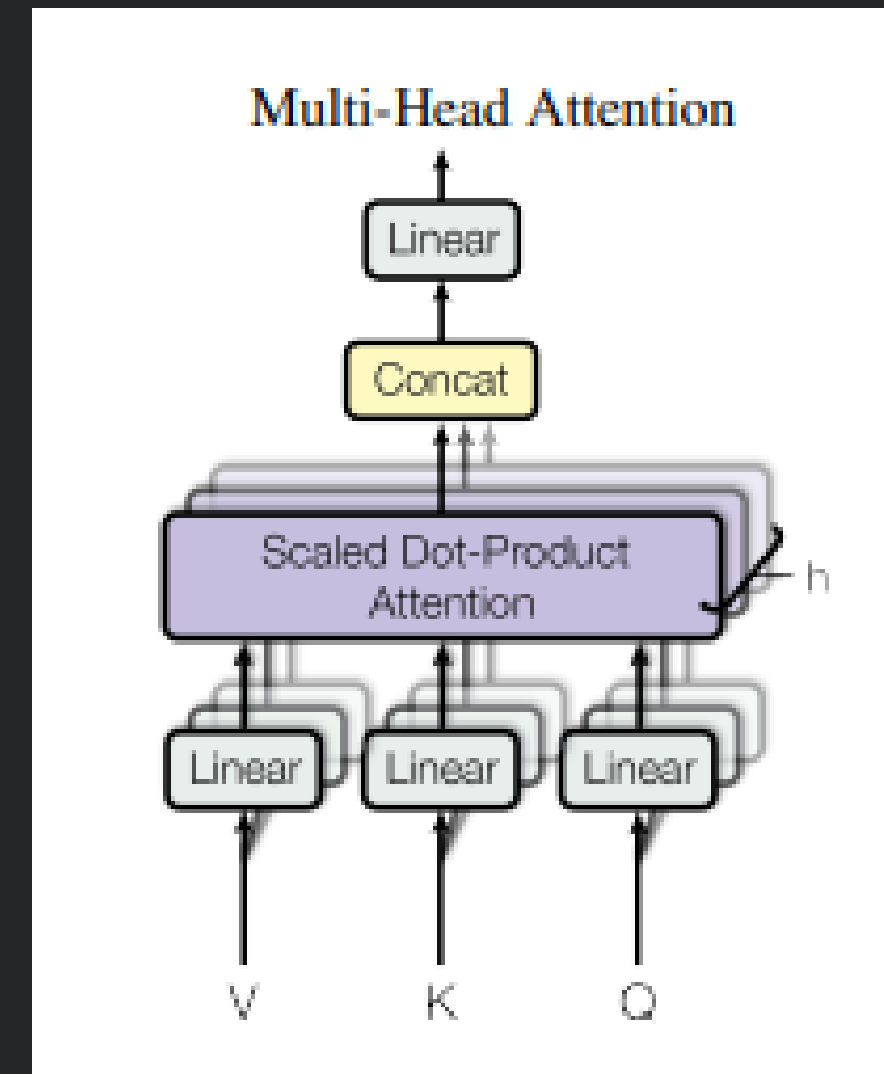
- On multiplie par la matrice W_v les embedding pour obtenir les values.
- $d(W_v) = d_model$
- En multipliant les SoftMaxs par V , on obtient les modifications à faire sur les embeddings.



TRANSFORMERS

Multi-Head Attention

- On répète l'attention $h = 8$ fois en parallèle
- Chaque tête apprend une relation différente
- Combine en concaténation puis projection finale



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

TRANSFORMERS

Comparaison

- RNN : séquentiel $O(n)$
- CNN : plusieurs couches pour connecter
loin
- Self-Attention : toutes positions
connectées en 1 opération
- Complexité par couche = $O(n^2 \cdot d)$ mais
séquentiel = $O(1)$
- Meilleur pour apprendre dépendances
longues
- n : longueur de la séquence

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

TRANSFORMERS

Mise en place des expérience

- Transformer 12H
- Transformers (big) : 3.5 jours
- 8 P100 Nvidia GPU
- Critère BLEU (bilingual evaluation understudy)

Explication BLEU :

n-gram : est le nombre de paquet de pris est comparer

Exemple : “Bonjour mon nom est Edouard” et “Bonjour je m’appelle Edouard”

TRANSFORMERS

Résultats après entraînement

- EN→DE : 28.4 BLEU
- EN→FR : 41.8 BLEU
- Entraînement moins coûteux que
ConvS2S/GNMT
- Transformer 12H
- Transformers (big) : 3.5 jours

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

TRANSFORMERS

La différence entre le BIG et le normal

Un jeu de dimension et autre:

-Dmodel : dimension des embedding

-dk et dv dimension des vecteurs keys and
valeur (resp.)

-Pdrop (Le taux de “d’oubli”
d’information)

-Training steps

-PPL (Perplexity) (Incertitude du model)

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	parangs $\times 10$	
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65	
(A)					1	512	512			5.29	24.9		
					4	128	128			5.00	25.5		
					16	32	32			4.91	25.8		
					32	16	16			5.01	25.4		
(B)					16					5.16	25.1	58	
					32					5.01	25.4	60	
(C)	2									6.11	23.7	36	
	4									5.19	25.3	50	
	8									4.88	25.5	80	
		256			32	32			5.75	24.5	28		
		1024			128	128			4.66	26.0	168		
			1024							5.12	25.4	53	
(D)			4096							4.75	26.2	90	
							0.0			5.77	24.6		
							0.2			4.95	25.5		
								0.0		4.67	25.3		
(E)								0.2		5.47	25.7		
	positional embedding instead of sinusoids									4.92	25.7		
big	6	1024	4096	16					0.3	300K	4.33	26.4	213

TRANSFORMERS

Visualisation

